TwinMapper

Development Phases

Module grouping, dependency order, and entry/exit conditions for all 10 phases

Version 1.0 • March 2026

Table of Contents

Table of Contents 2
Overview 3
Architectural Sections 4
Build-Time Pipeline (Phases 1–3) 4
Runtime Platform (Phases 4–6) 4
Consumption Layer (Phases 7–10) 4
Dependency Spine 6
Module Dependency Tree 6
Phase Details 8
Build-Time Pipeline 8
Runtime Platform 10
Consumption Layer 12
Phase Summary 15
Guiding Principles 16
Deferred and Non-Goals 16
Deferred — Post-V1 16
Hard Non-Goals 16

Overview

This document defines the development phases for TwinMapper. Phases are ordered by dependency, stability risk, and integration readiness. Each phase has a clear entry condition, a defined scope, and a clear exit condition before the next phase begins.

The 14 core modules are organized into 10 phases across three architectural sections: the build-time pipeline, the runtime platform, and the consumption layer. This distinction is the organizing principle — it keeps the dependency story clean and avoids mixing authoring, codegen, runtime, and tooling concerns.

Prerequisite

Two open implementation decisions must be resolved before Phase 1 begins: (1) Source writer tool for twinmapper-codegen, and (2) DefinitionSet runtime identity mechanism.

Architectural Sections

The development flow distinguishes three sections. This separation makes the dependency story explicit and prevents modules from crossing architectural boundaries.

Build-Time Pipeline (Phases 1–3)

The foundation, definition readers, and code generation engine. This is where TwinMapper's primary value lives — definition-driven generated mapping. These phases produce the generated artifacts that everything downstream depends on.

Runtime Platform (Phases 4–6)

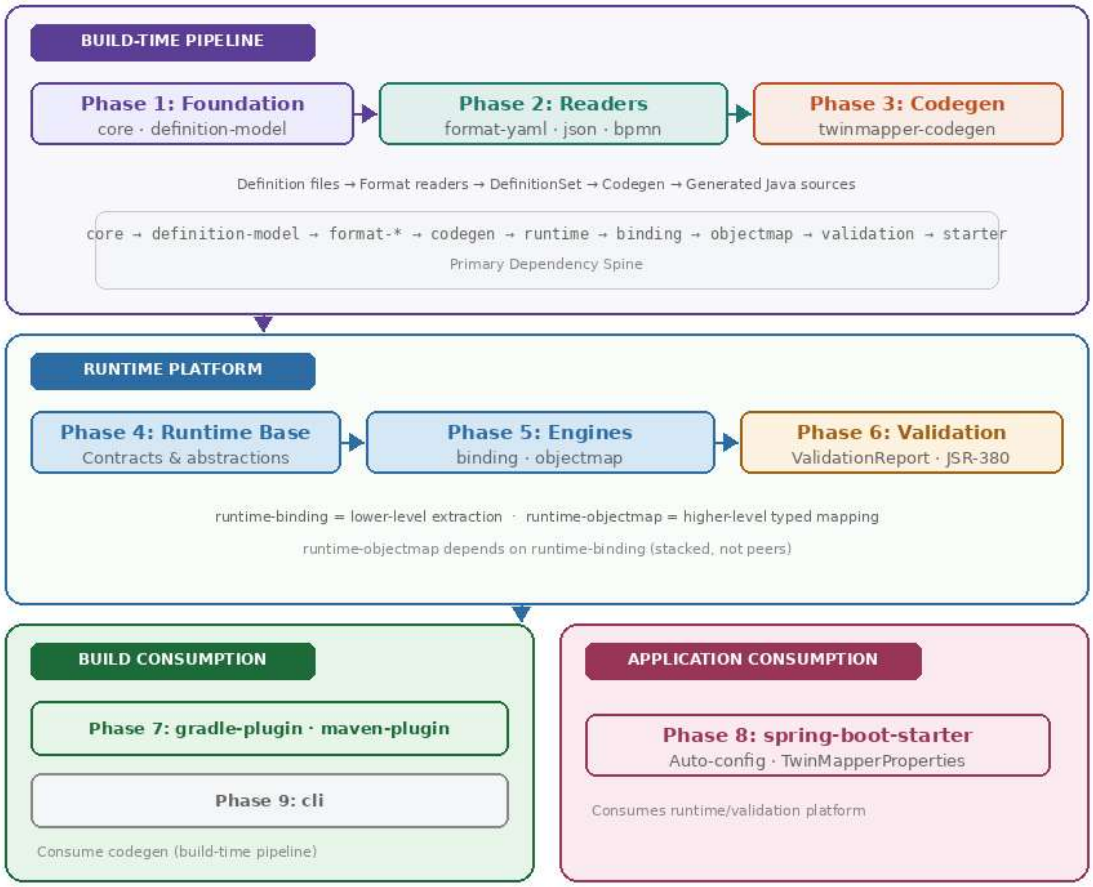
The shared runtime contracts, document binding engine, object mapping engine, and validation engine. These phases execute the generated code at runtime. The runtime platform is layered: runtime-binding is lower-level extraction/traversal infrastructure; runtime-objectmap is the higher-level typed mapping layer built on top of that foundation.

Consumption Layer (Phases 7–10)

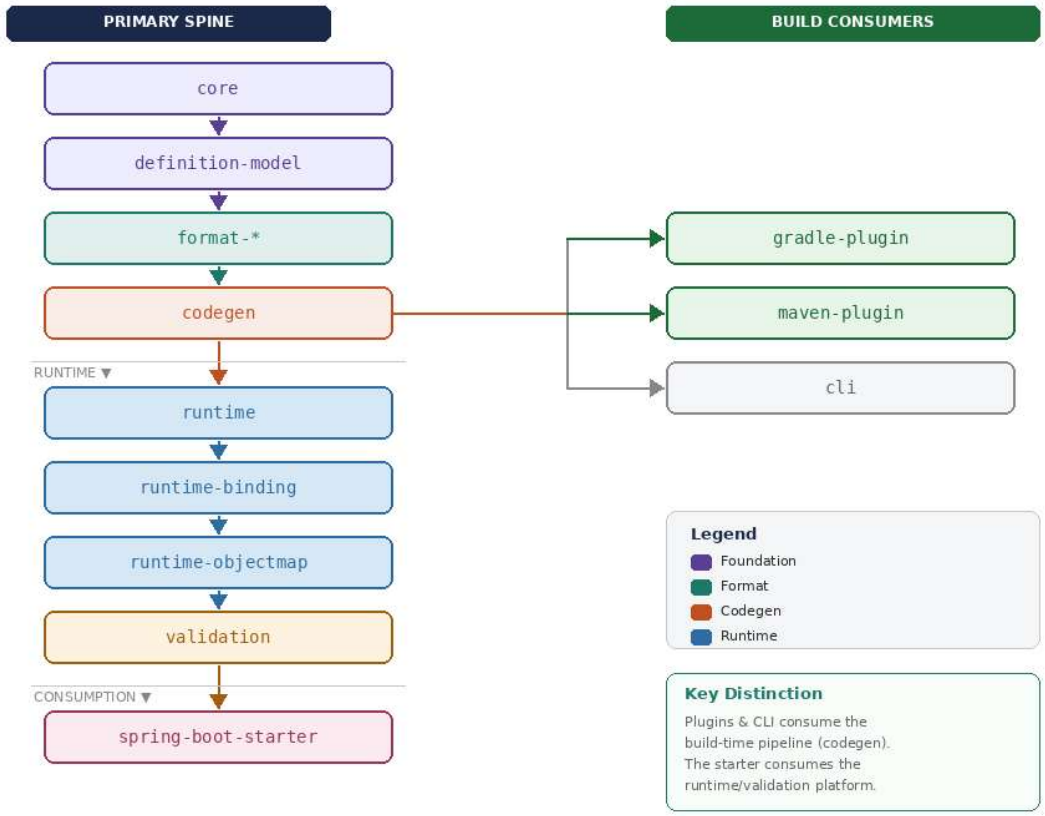
The build plugins, Spring Boot starter, CLI, and optional modules. These are delivery channels that expose TwinMapper to build systems and applications. They are separated into two sub-layers:

Consumption Layer A — Build Consumption: gradle-plugin, maven-plugin, cli. These consume the build-time pipeline (codegen).

Consumption Layer B — Application Consumption: spring-boot-starter. This consumes the runtime/validation platform.


BUILD-TIME PIPELINE
Phase 1: Foundation
core · definition-model
Phase 2: Readers
format-yaml · json · bpmn
Phase 3: Codegen
twinmapper-codegen
Definition files → Format readers → DefinitionSet → Codegen → Generated Java sources
core → definition-model → format-* → codegen → runtime → binding → objectmap → validation → starter
Primary Dependency Spine
RUNTIME PLATFORM
Phase 4: Runtime Base
Contracts & abstractions
Phase 5: Engines
binding · objectmap
Phase 6: Validation
ValidationReport · JSR-380
runtime-binding = lower-level extraction · runtime-objectmap = higher-level typed mapping
runtime-objectmap depends on runtime-binding (stacked, not peers)
BUILD CONSUMPTION
Phase 7: gradle-plugin · maven-plugin
Phase 9: cli
Consume codegen (build-time pipeline)
APPLICATION CONSUMPTION
Phase 8: spring-boot-starter
Auto-config · TwinMapperProperties
Consumes runtime/validation platform

Dependency Spine



Module Dependency Tree

BUILD-TIME DEPENDENCIES

```
twinmapper-core
└── twinmapper-definition-model
│   ├── twinmapper-codegen
│   ├── twinmapper-format-json
│   ├── twinmapper-format-yaml
│   └── twinmapper-format-bpmn
```

RUNTIME DEPENDENCIES

```
twinmapper-runtime  (shared-base)
├── twinmapper-runtime-binding
│   ├── twinmapper-format-json
│   ├── twinmapper-format-yaml
│   └── twinmapper-format-bpmn
└── twinmapper-runtime-objectmap

twinmapper-validation
└── twinmapper-runtime-binding  (error model)
```

CONSUMPTION DEPENDENCIES

```
twinmapper-spring-boot-starter
├── twinmapper-runtime-binding
├── twinmapper-runtime-objectmap
└── twinmapper-validation

twinmapper-gradle-plugin
└── twinmapper-codegen

twinmapper-maven-plugin
└── twinmapper-codegen

twinmapper-cli
```

Phase Details

Build-Time Pipeline

PHASE 1 BUILD-TIME PIPELINE

Foundation

Goal: Establish the shared foundation and the canonical internal meta-model that all other phases depend on.

Modules

- `twinmapper-core`
- `twinmapper-definition-model`

Scope

- Common error model and result model
- Shared constants and base exception hierarchy
- Base SPI interfaces and shared diagnostics structures
- Path and reference abstractions, shared policy enums
- Spring utility integration: Assert, StringUtils, ObjectUtils, ClassUtils, AnnotationUtils, ReflectionUtils
- Resource-loading: ResourceUtils, PathMatchingResourcePatternResolver
- Deterministic SPI ordering: OrderComparator, PriorityOrdered, @Order, Ordered
- All meta-model types: DefinitionSet, TypeDefinition, ObjectTypeDefinition, EnumTypeDefinition, FieldDefinition, CollectionDefinition, MapDefinition, AliasDefinition, ConstraintDefinition, and all others
- Deterministic DefinitionSet runtime identity mechanism
- Format-neutral design with no Spring container dependency

Entry: Open implementation decisions resolved.

Exit: core and definition-model compile cleanly. All meta-model types defined and unit-tested. No downstream module depends on anything outside these two.

PHASE 2 BUILD-TIME PIPELINE

Definition Readers

Goal: Enable all three formats to read definition files into the internal meta-model at build time.

Modules

- `twinmapper-format-yaml`
- `twinmapper-format-json`
- `twinmapper-format-bpmn`

Scope

- YAML definition reader using SnakeYAML with mandatory SafeConstructor
- JSON definition reader using Jackson ObjectMapper
- BPMN definition reader using JDK StAX with XXE disabled and DTD disabled
- Each reader normalizes format-specific constructs into canonical DefinitionSet types
- Runtime NodeCursor implementation per format for document binding
- Security tests verifying XXE rejection and safe YAML construction

Entry: Phase 1 complete and verified.

Exit: All three readers parse sample definitions into identical DefinitionSet structures. Round-trip consistency verified. Security tests pass.

PHASE 3 BUILD-TIME PIPELINE

Code Generation

Goal: Generate all Java source artifacts from the internal definition model.

Modules

- `twinmapper-codegen`

Scope

- Generate DTOs (immutable records by default; mutable when options require)
- Generate enums from declared values
- Generate metadata descriptor classes
- Generate document binder classes for YAML, JSON, and BPMN
- Generate schema validator classes
- Generate registry classes (binder, mapper, metadata, converter, type lookup)
- Generate object mapper classes (create, update, patch, inverse)
- Generate collection and nested mappers
- Lombok-free generated output
- Optional @Component on generated beans when Spring bean mode enabled
- @Qualifier, @Primary, @Conditional for bean disambiguation
- basePackage configuration for consuming application alignment

Entry: Phase 2 complete and verified.

Exit: A representative definition set generates correct, compilable Java sources. Generated sources match the codegen contract. No runtime dependency in generated code.

Runtime Platform

PHASE 4 RUNTIME PLATFORM

Runtime Base

Goal: Establish the shared runtime contract layer that both runtime engines depend on.

Modules

- `twinmapper-runtime`

Scope

- TwinMapperRuntime facade interface
- BindingContext and MappingContext
- Registry interfaces for binders, mappers, converters, and metadata
- AliasResolutionPolicy shared contract
- CompatibilityMode enum (STRICT, COMPATIBLE, LENIENT)
- TwinMapperRuntimeConfigurer interface for programmatic customization
- Common runtime policies and result models
- Shared diagnostic transport models

Entry: Phase 3 complete and verified.

Exit: All runtime contract interfaces defined and stable. No binding logic or object-mapping implementations present. Pure Java contracts only.

PHASE 5 RUNTIME PLATFORM

Runtime Engines

Goal: Implement the document binding engine and the typed object mapping engine.

Modules

- `twinmapper-runtime-binding`
- `twinmapper-runtime-objectmap`

Scope

- runtime-binding: NodeCursor traversal abstraction and dispatch to generated binders
- runtime-binding: BindingResult accumulation with path-aware diagnostics
- runtime-binding: Required fields, defaults, alias resolution, strict unknown-field behavior
- runtime-binding: YAML, JSON, and BPMN document binding operational
- runtime-objectmap: Generated create, update, patch, and inverse mapper execution
- runtime-objectmap: TwinObjectMapperRegistry for mapper lookup
- runtime-objectmap: Spring ConversionService as converter backend
- runtime-objectmap: BeanWrapper for UPDATE/PATCH on Spring-managed bean targets
- runtime-objectmap: Proxy-safe reflection via AopUtils, AopProxyUtils, ProxyUtils
- runtime-objectmap: All 13 supported mapping flows operational
- runtime-objectmap: Null-policy enforcement, reusable profiles, flattening, expansion

Entry: Phase 4 complete and verified.

Exit: Document binding works end-to-end for all three formats. All 13 mapping flows work with generated mappers. Proxy-safe behavior verified.

PHASE 6 RUNTIME PLATFORM

Validation

Goal: Enable the full validation pipeline with Spring-native integration.

Modules

- `twinmapper-validation`

Scope

- Generated schema validators executed against populated DTOs
- ValidationReport with path-aware aggregated error reporting
- All constraint types: required, optional, default, enum, forbidden, deprecated, conditional, one-of, mutually exclusive, min, max, minLength, maxLength, regex/pattern, unknown-field
- Spring Validator, SmartValidator, Errors, BindingResult integration
- ConstraintValidatorFactory for Spring-injected custom constraint validators
- @Valid, @Validated, JSR-380 support
- Handwritten semantic validators supported alongside generated schema validation

Entry: Phase 5 complete and verified.

Exit: Validation errors accumulate into ValidationReport and translate cleanly into Spring MVC exception types. JSR-380 annotations work alongside generated validators.

Consumption Layer

Build Consumption vs Application Consumption

Plugins and CLI consume the build-time pipeline (codegen). The Spring Boot starter consumes the runtime/validation platform. These are architecturally distinct delivery channels with different dependency directions.

PHASE 7 BUILD CONSUMPTION

Build Plugins

Goal: Integrate code generation into Gradle and Maven build lifecycles.

Modules

- `twinmapper-gradle-plugin`
- `twinmapper-maven-plugin`

Scope

- Gradle: Scan definition directories, validate, invoke codegen
- Gradle: Register generated sources via sourceSets.main.java.srcDir(generatedDir)
- Gradle: Plugin API and Worker API, deterministic CI-friendly output
- Gradle: Dry-run, diagnostics, IDE-compatible source root registration
- Maven: Bind to generate-sources lifecycle phase
- Maven: Register generated sources via project.addCompileSourceRoot(generatedDir)
- Maven: Validation-only and dry-run goals, IDE-compatible source roots

Entry: Phase 3 (codegen) complete and stable.
Exit: Spring Boot project generates correct Java sources with gradle build and mvn compile. IDE tooling recognizes the generated source directory.

PHASE 8 APPLICATION CONSUMPTION

Spring Boot Starter

Goal: Provide a first-class Spring Boot auto-configuration experience.

Modules

- `twinmapper-spring-boot-starter`

Scope

- @AutoConfiguration with AutoConfiguration.imports (not spring.factories)
- TwinMapperProperties via @ConfigurationProperties(prefix="twinmapper")
- Conditional bean registration for runtime-binding, runtime-objectmap, and validation
- ConversionService integration wiring
- Actuator health indicator and metrics endpoints
- Optional feature flag toggles via properties
- spring-configuration-metadata.json for IDE auto-completion
- @AutoConfigureTwinMapper test annotation
- GraalVM native image baseline verification

Entry: Phases 4–6 (runtime platform) complete and verified.
Exit: Spring Boot application auto-configures TwinMapper. Properties bind correctly. Actuator endpoints verified.

PHASE 9 BUILD CONSUMPTION

CLI

Goal: Provide developer tooling for definition authoring and debugging.

Modules

- `twinmapper-cli`

Scope

- Validate definition files offline
- Dry-run code generation without writing output
- Inspect and report on normalized definition model
- Definition linting and diagnostics
- Optional: sample-to-definition assistant (CLI flag only, requires human review)
- SpringApplication.setWebApplicationType(WebApplicationType.NONE) mandatory
- ApplicationRunner, CommandLineRunner, ExitCodeGenerator as CLI contracts

Entry: Phases 3 (codegen) and 6 (validation) complete.

Exit: CLI validates definitions, runs dry-run generation, and reports diagnostics correctly. No embedded servlet container starts.

PHASE 10 OPTIONAL

Optional Extensions

Goal: Implement all property-gated optional features. None may replace the generated primary path.

Modules

- `twinmapper-annotations (optional)`
- `twinmapper-annotation-processor (optional)`
- `twinmapper-runtime-compat (optional)`

Scope

- Convention-based mapping mode (twinmapper.objectmap.convention-mapping.enabled=false)
- Annotation-heavy usage mode (twinmapper.annotations.enabled=false)
- Reflection-based compatibility helpers (twinmapper.objectmap.reflection-compat.enabled=false)
- Bounded runtime discovery helpers (twinmapper.binding.bounded-discovery.enabled=false)
- Alias/fallback compatibility resolution (twinmapper.compatibility-resolution.enabled=false)
- Sample-to-definition assistant tooling (CLI flag only)
- All features: property-gated, default off, opt-in only, never primary path

Entry: All core phases (1–9) complete.

Exit: Each optional feature activates cleanly when its property is set to true. No optional feature affects the generated primary path.

Phase Summary

Phase	Section	Modules	Goal
1	Build-Time	core, definition-model	Foundation and meta-model
2	Build-Time	format-yaml, format-json, format-bpmn	Definition readers
3	Build-Time	codegen	Code generation
4	Runtime	runtime	Runtime contracts
5	Runtime	runtime-binding, runtime-objectmap	Runtime engines
6	Runtime	validation	Validation engine
7	Build Consumption	gradle-plugin, maven-plugin	Build plugins
8	App Consumption	spring-boot-starter	Spring Boot integration
9	Build Consumption	cli	Developer tooling
10	Optional	annotations, annotation-processor, runtime-compat	Optional extensions

Guiding Principles

TwinMapper is build-time-first.
Its primary path is definition-driven code generation, not reflection-first runtime mapping.

BPMN is an authoring format, not a workflow engine.
BPMN definitions are parsed into the canonical meta-model and do not define TwinMapper runtime semantics.

The runtime platform is layered.
runtime-binding handles traversal and extraction; runtime-objectmap handles typed object mapping on top of that foundation. runtime-objectmap depends on runtime-binding, not the other way around.

Plugins, starter, and CLI are delivery channels.
They expose TwinMapper to build systems and applications, but are not part of the core engine.

Deferred and Non-Goals

Deferred — Post-V1

These items are explicitly out of scope for all phases above.

Item	Status
Union/discriminator-capable types	Deferred until base DSL, codegen, and binder are stable
JSON-based object-mapping DSL	YAML DSL is the only mapping DSL in V1
Additional definition DSL formats	New formats addable via SPI in future releases
Annotation-first architecture	Annotations remain optional convenience only in V1

Hard Non-Goals

Non-Goal	Reason
Uncontrolled live schema learning	Contradicts compile-time-first principle
Silent hidden guessing	All mapping must be explicit and auditable

Runtime-first architecture	Compile-time-first is locked
Domain-specific semantics in core	Core platform remains generic